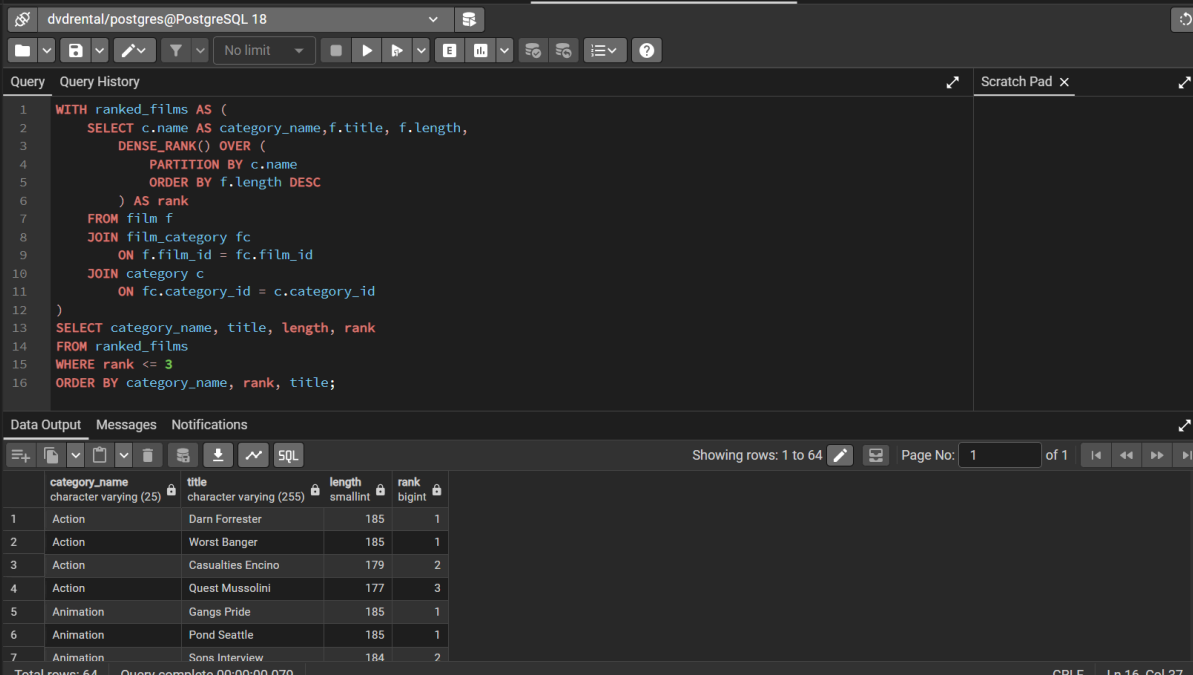


Lab 1:



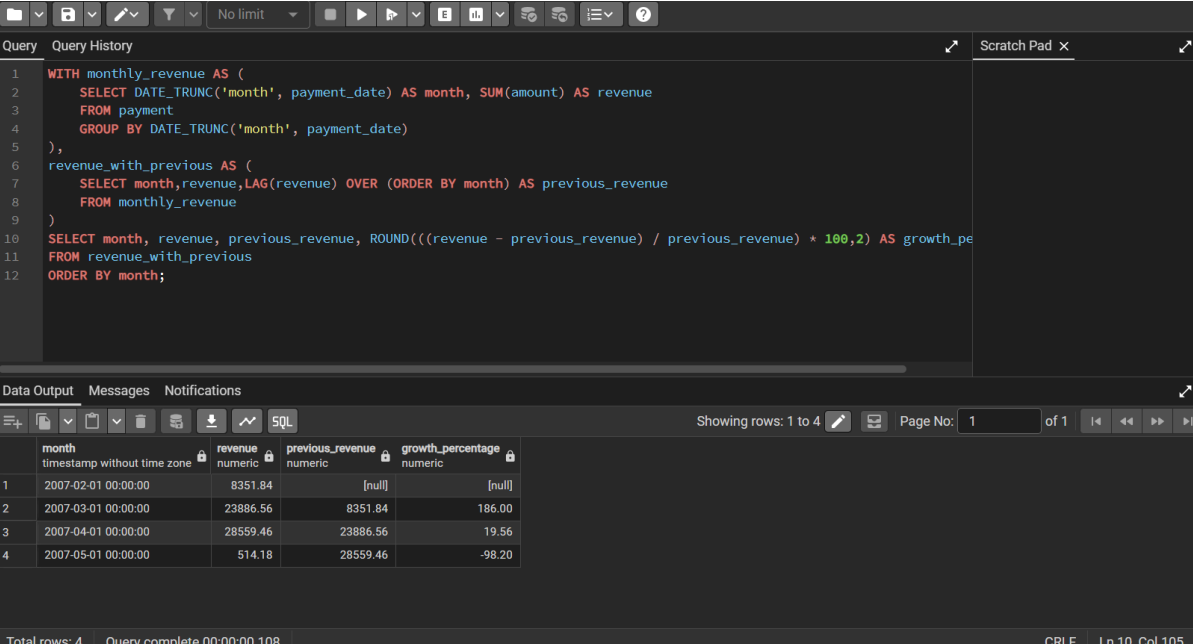
```
1 WITH ranked_films AS (  
2     SELECT c.name AS category_name, f.title, f.length,  
3         DENSE_RANK() OVER (  
4             PARTITION BY c.name  
5             ORDER BY f.length DESC  
6         ) AS rank  
7     FROM film f  
8     JOIN film_category fc  
9         ON f.film_id = fc.film_id  
10    JOIN category c  
11        ON fc.category_id = c.category_id  
12 )  
13 SELECT category_name, title, length, rank  
14 FROM ranked_films  
15 WHERE rank <= 3  
16 ORDER BY category_name, rank, title;
```

	category_name	title	length	rank
1	Action	Damn Forrester	185	1
2	Action	Worst Banger	185	1
3	Action	Casualties Encino	179	2
4	Action	Quest Mussolini	177	3
5	Animation	Gangs Pride	185	1
6	Animation	Pond Seattle	185	1
7	Animation	Sons Interview	184	2

Total rows: 64 Query complete 00:00:00.079 CRLF Ln 16, Col 37

The query joins the film, film_category, and category tables and uses DENSE_RANK() to rank films by length within each category. PARTITION BY ensures that the ranking starts again for every category.

Lab 2:



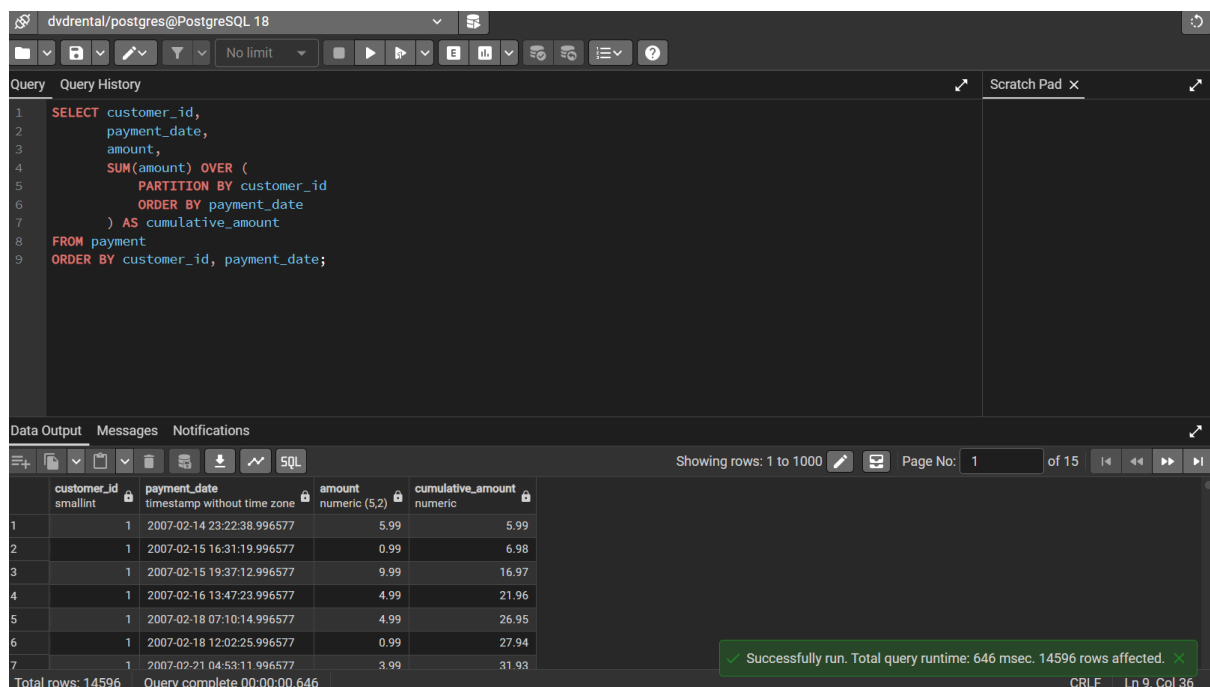
```
1 WITH monthly_revenue AS (  
2     SELECT DATE_TRUNC('month', payment_date) AS month, SUM(amount) AS revenue  
3     FROM payment  
4     GROUP BY DATE_TRUNC('month', payment_date)  
5 ),  
6 revenue_with_previous AS (  
7     SELECT month, revenue, LAG(revenue) OVER (ORDER BY month) AS previous_revenue  
8     FROM monthly_revenue  
9 )  
10 SELECT month, revenue, previous_revenue, ROUND(((revenue - previous_revenue) / previous_revenue) * 100, 2) AS growth_percentage  
11 FROM revenue_with_previous  
12 ORDER BY month;
```

	month	revenue	previous_revenue	growth_percentage
1	2007-02-01 00:00:00	8351.84	[null]	[null]
2	2007-03-01 00:00:00	23886.56	8351.84	186.00
3	2007-04-01 00:00:00	28559.46	23886.56	19.56
4	2007-05-01 00:00:00	514.18	28559.46	-98.20

Total rows: 4 Query complete 00:00:00.108 CRLF Ln 10, Col 105

The query calculates total revenue for each month using DATE_TRUNC() and SUM(). LAG() retrieves the previous month's revenue, which is then used to calculate the Month-over-Month growth rate.

Lab 3:



The screenshot shows a PostgreSQL IDE window titled 'dvdrental/postgres@PostgreSQL 18'. The query editor contains the following SQL code:

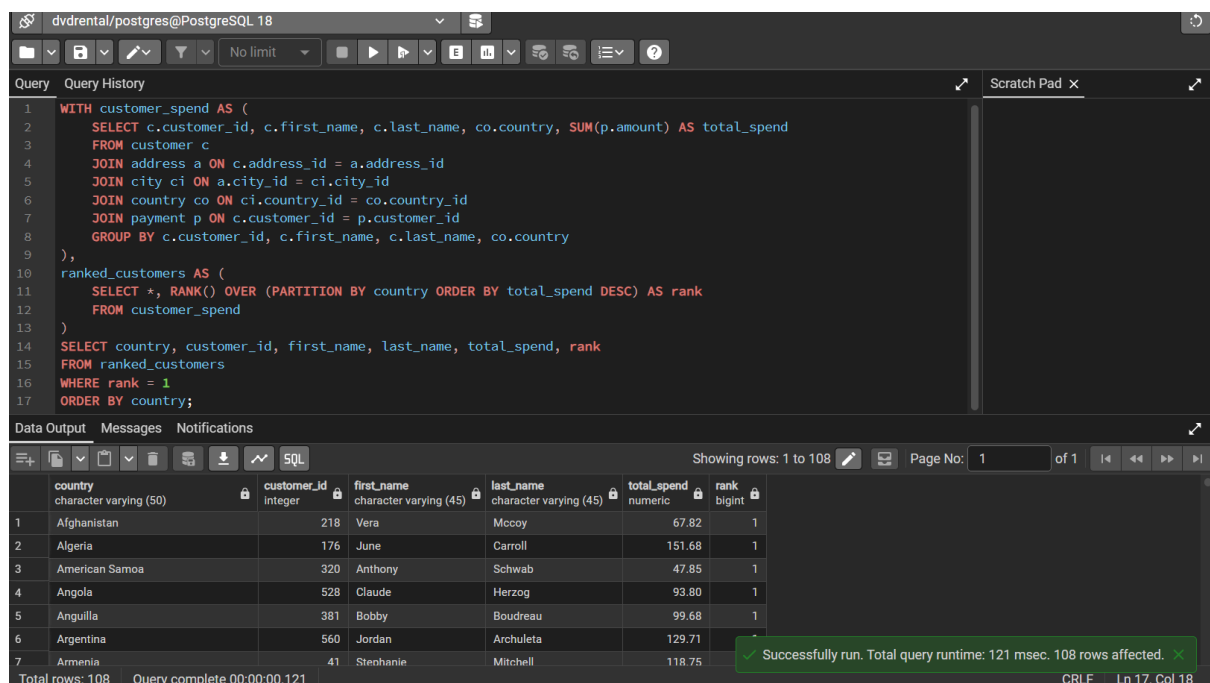
```
1 SELECT customer_id,
2        payment_date,
3        amount,
4        SUM(amount) OVER (
5          PARTITION BY customer_id
6          ORDER BY payment_date
7        ) AS cumulative_amount
8 FROM payment
9 ORDER BY customer_id, payment_date;
```

The 'Data Output' tab is active, displaying the results of the query. The table has 5 columns: customer_id, payment_date, amount, and cumulative_amount. The results show 14596 rows, with the first 7 rows visible. A green status bar at the bottom indicates 'Successfully run. Total query runtime: 646 msec. 14596 rows affected.'

customer_id	payment_date	amount	cumulative_amount
1	2007-02-14 23:22:38.996577	5.99	5.99
1	2007-02-15 16:31:19.996577	0.99	6.98
1	2007-02-15 19:37:12.996577	9.99	16.97
1	2007-02-16 13:47:23.996577	4.99	21.96
1	2007-02-18 07:10:14.996577	4.99	26.95
1	2007-02-18 12:02:25.996577	0.99	27.94
1	2007-02-21 04:53:11.996577	3.99	31.93

The query uses SUM() OVER() to calculate the cumulative payment amount for each customer. The transactions are ordered by payment_date, so the cumulative value increases chronologically after each payment.

Lab 4 :



The screenshot shows a PostgreSQL IDE window titled 'dvdrental/postgres@PostgreSQL 18'. The query editor contains the following SQL code:

```
1 WITH customer_spend AS (
2   SELECT c.customer_id, c.first_name, c.last_name, co.country, SUM(p.amount) AS total_spend
3   FROM customer c
4   JOIN address a ON c.address_id = a.address_id
5   JOIN city ci ON a.city_id = ci.city_id
6   JOIN country co ON ci.country_id = co.country_id
7   JOIN payment p ON c.customer_id = p.customer_id
8   GROUP BY c.customer_id, c.first_name, c.last_name, co.country
9 ),
10 ranked_customers AS (
11   SELECT *, RANK() OVER (PARTITION BY country ORDER BY total_spend DESC) AS rank
12   FROM customer_spend
13 )
14 SELECT country, customer_id, first_name, last_name, total_spend, rank
15 FROM ranked_customers
16 WHERE rank = 1
17 ORDER BY country;
```

The 'Data Output' tab is active, displaying the results of the query. The table has 6 columns: country, customer_id, first_name, last_name, total_spend, and rank. The results show 108 rows, with the first 7 rows visible. A green status bar at the bottom indicates 'Successfully run. Total query runtime: 121 msec. 108 rows affected.'

country	customer_id	first_name	last_name	total_spend	rank
Afghanistan	218	Vera	Mccoy	67.82	1
Algeria	176	June	Carroll	151.68	1
American Samoa	320	Anthony	Schwab	47.85	1
Angola	528	Claude	Herzog	93.80	1
Anguilla	381	Bobby	Boudreau	99.68	1
Argentina	560	Jordan	Archuleta	129.71	1
Armenia	41	Stenhanie	Mitchell	118.75	1

The query first calculates each customer's total spending and country, then uses RANK() to rank customers within each country. The CTE allows us to filter for rank = 1 and obtain the highest-spending customer from each country.